INSTITUTO FEDERAL
São Paulo
Câmpus São Carlos

APR1 – Algoritmos e Programação 1
Manipulação de Strings

Profa. Dra. Eloize Seno

INSTITUTO FEDERAL
São Paulo
Câmpus São Carlos

Strings

- Uma string (=tipo str) é uma sequência (cadeia) de caracteres
- Podemos representar uma string colando-a entre aspas (“”) ou apóstrofes (‘’)
- O uso desses 2 marcadores é útil quando queremos utilizar um deles dentro da string, como nos exemplos:

```
texto = "Esta string usa 'aspas' como demarcador"
print(texto)
Saída: Esta string usa 'aspas' como demarcador

novo_texto = 'Esta string usa "apóstrofes" como demarcador'
print(novo_texto)
Saída: Esta string usa "apóstrofes" como demarcador
```

Strings

- Uma cadeia que não possui nenhum caractere, chamada de **string vazia**, é também uma string

- A **string vazia** é representada usando apóstrofes (‘’) ou aspas (“”)

- Uma string com um ou mais espaços em branco, é diferente de uma string vazia

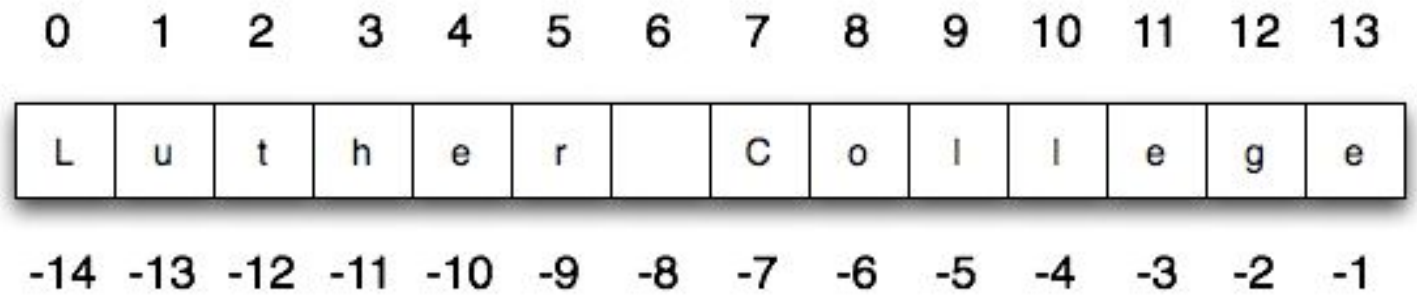
Strings: criação

- Assim como listas:
  - um caractere de uma string pode ser acessado por meio de índice
  - o comprimento pode ser dado pela função len()
  - os caracteres de uma string podem ser fatiados e concatenados
  - Exemplos:

```
frase = ""   # string vazia
frase = frase + "Essa string usa "
meio  = '"apóstrofes" '
fim   = "como demarcador. "
frase = frase + meio + fim
print("A frase: ", frase)
print("Tem comprimento: ", len(frase))
```

Strings: indexação

- Assim como os elementos de uma lista, os caracteres de uma string podem ser indexados por um índice (um número inteiro) entre colchetes
- O **operador de indexação** seleciona um único caractere de uma string
- Os caracteres são acessados por sua posição ou valor do índice. Por exemplo, na sequência mostrada abaixo, os 14 caracteres são indexados da esquerda para a direita a partir posição 0 até a posição 13



- As posições podem ser acessadas também da direita para a esquerda usando números negativos, onde -1 é o índice mais à direita e assim por diante

Strings: indexação

- Teste o exemplo:

```
frase = "Eu posso ajudar respondendo perguntas no fórum!"
sorteio = [5, -4, 3, -8, 11]
acertos = 0
for i in sorteio:
    resposta = input(f"Qual o caractere de índice {i} ")
    if frase[i] == resposta:
       print("Parabéns, você acertou!")
       acertos+=1
    else:
       print(f"Você errou. O caractere de índice {i} é:
{frase[i]}")
print(f"Você acertou {acertos} de {len(sorteio)} perguntas.")
```

Strings: fatiamento

- Assim como listas uma string também pode ser fatiada e o intervalo definido por início e fim é fechado à esquerda e aberto (não incluso) à direita
- Teste o exemplo:

```
ini, fim = 3, 7
texto = "Eu posso ajudar respondendo perguntas no fórum!"
fatia = texto[ ini : fim ]
print("Seja o texto: ", texto)
resposta = input(f"Digite a string que corresponde à fatia texto[{ini}:{fim}] = ")
if resposta == fatia:
        print("Muito bem, você acertou!!")
else:
        print("Você escreveu         : ", resposta)
        print("mas o valor correto é: ", fatia)
```

Strings: fatiamento

- Teste o exemplo:

```
ini, fim, passo = 3, 13, 2
texto = "Eu posso ajudar respondendo perguntas no fórum!"
fatia = texto[ ini : fim : passo ]
resposta = input(f"Digite a string que corresponde à fatia texto[{ini}:{fim}:{passo}] = ")
if resposta == fatia:
    print("Muito bem, você acertou!!")
else:
    print("Você escreveu         : ", resposta)
    print("mas o valor correto é: ", fatia)
```

Strings não são listas

- Apesar de serem criadas usando um par de ” (aspas) ou ‘ (apóstrofos) como demarcadores, a função *len()*, a concatenação, a indexação e o fatiamento de strings são realizados de forma semelhante às listas
- A grande diferença é que strings são **imutáveis**, ou seja, não é possível alterar os seus elementos
- Teste o exemplo:

```
texto = "Essa é uma 'string'"
print(texto)
texto[2] = 'S'
print(texto)
```

Strings não são listas

- Como strings são IMUTÁVEIS, caso você precise de uma string diferente, é necessário criar uma nova utilizando, caso desejar, fatias (partes) de outras strings

- Exemplo:

```
texto = "Essa é uma 'string'"
print(texto)
novo_texto = 'Mais' + texto[6:]
print(novo_texto)
```

Comparação de Strings

- Os operadores de comparação também funcionam com strings
- Para ver se duas strings são iguais, basta escrever uma expressão booleana usando o operador de igualdade. Exemplo:

```
word = "banana"
if word == "banana":
    print("Yes, we have bananas!")
else:
    print("Yes, we have NO bananas!")
```

- Outras operações de comparação são úteis para colocar palavras em *ordem lexicográfica,* isto é semelhante à ordem alfabética usada em um dicionário, exceto que todas as letras maiúsculas vêm antes de todas as letras minúsculas.

Comparação de Strings

- Teste os exemplos:

```
word = "zebra"
if word < "banana":
    print("Your word, " + word + ", comes before banana.")
elif word > "banana":
    print("Your word, " + word + ", comes after banana.")
else:
    print("Yes, we have no bananas!")

print("apple" < "banana")
print("apple" == "Apple")
print("apple" < "Apple")
```

Comparação de Strings

- Letras maiúsculas e minúsculas são consideradas diferentes umas das outras
- A forma como o computador sabe que eles são diferentes é que cada caractere corresponde a um valor inteiro único. Por exemplo, “A” é de 65, “B” é de 66, e “5” é 53
- É possível descobrir o chamado valor ordinal de um caractere utilizando a função ord:

```
print(ord("A"))
print(ord("B"))
print(ord("5"))

print(ord("a"))
print("apple" > "Apple")
```

Comparação de Strings

- Em uma comparação de caracteres ou strings, o Python converte os caracteres para seus valores ordinais e compara os inteiros da esquerda para a direita, por isso que no exemplo anterior, “a” é maior que “A” de forma que “apple” é maior do que “Apple”

- Nós geralmente ignoramos a capitalização ao comparar duas palavras. No entanto, os computadores não. Uma maneira comum de resolver este problema é converter as strings para um formato padrão, tal como todas as letras minúsculas, antes de realizar a comparação

Varredura com while

- O laço while também pode ser usado para percorrer uma string

```
fruta = "apple"
i = 0
while i < len(fruta):
    print(fruta[i])
    i += 1
```

Varredura com for

- O laço for pode ser usado para iterar sobre os caracteres de uma string (cada posição dentro da string)
- Exemplo:

```
fruta = "apple"
for i in range(len(fruta)):
    atual = fruta[i]
    print(atual)
```

Operadores in e not in

- Uma string é uma substring de si mesmo, e a string vazia é uma substring de qualquer outra string

- O operador **not in** retorna o resultado lógico oposto de **in**

- Teste o exemplo:

```
print('a' in 'a')
print('apple' in 'apple')
print('' in 'a')
print('' in 'apple')
```

Operadores in e not in

- Outro exemplo: dada uma palavra, retirar suas vogais e mostra a nova string

```
palavra=input("Digite uma palavra: ")
vogais = "aeiouAEIOU"
str_sem_vogais = ""
for cada_letra in palavra:
    if cada_letra not in vogais:
        str_sem_vogais = str_sem_vogais + cada_letra
print(str_sem_vogais)
```

Repetições e Contagens

- Neste exemplo o programa conta o número de vezes que uma letra em particular aparece em uma string:

```
palavra=input("Digite uma palavra: ")
caractere=input("Digite um caractere a ser contado: ")
contador = 0
for letra in palavra:
    if letra == caractere:
            contador = contador + 1
print(f"A letra {caractere} aparece {contador} vezes na
palavra {palavra}")
```

Repetições e Contagens

- Neste exemplo o programa mostra a posição da primeira ocorrência de uma letra em particular em uma string:

```
palavra=input("Digite uma palavra: ")
caractere=input("Digite um caractere a ser encontrado: ")
indice = 0
existe_letra = False
while indice < len(palavra) and not existe_letra:
    if palavra[indice] == caractere:
            existe_letra = True
    else:
            indice = indice + 1
if existe_letra:
    print(f"A primeira ocorrência do caractere {caractere} na palavra {palavra} está na posição {indice}")
else:
    print(f"Não há ocorrência do caractere {caractere} na palavra {palavra}!")
```

Classificação de Caracteres

- Muitas vezes, é útil examinar um caractere e testar se ele é maiúsculo ou minúsculo, ou se é uma letra ou um dígito
- O módulo string fornece várias constantes que são úteis para esses fins
- Um exemplo é a constante string.digits é equivalente a "0123456789", que pode ser usada para verificar se um caractere é um dígito usando o operador in
- A constante string.ascii_lowercase contém todas as letras ASCII que o sistema considera serem minúsculas
- Da mesma forma, string.ascii_uppercase contém todas as letras maiúsculas e string.punctuation contém todos os caracteres considerados símbolos de pontuação

Classificação de Caracteres

- Teste o código para verificar o conteúdo das constantes:

```
import string
print(string.ascii_lowercase)
print(string.ascii_uppercase)
print(string.digits)
print(string.punctuation)
```

Classificação de Caracteres

- O código abaixo verifica se o valor digitado é um número inteiro positivo e ilustra o uso da constante string.digits:

```
nro_ok=False
while not nro_ok:
    nro=input("Digite um número inteiro positivo: ")
    nro_ok=True
    for i in nro:
        if i not in string.digits:
            nro_ok=False
    if not nro_ok:
        print("O valor digitado não é um número inteiro
positivo!")
nro=int(nro) #faz a conversão
print("O valor digitado é: ",nro)
```

Classificação de Caracteres

- O código abaixo verifica se o valor digitado é um número inteiro positivo utilizando o método isdigit():

```
nro_ok=False
while not nro_ok:
    nro=input("Digite um número inteiro positivo: ")
    nro_ok=nro.isdigit()
    if not nro_ok:
        print("O valor digitado não é um número inteiro positivo!")

nro=int(nro) #faz a conversão
print("O valor digitado é: ",nro)
```

Métodos de Strings

- Strings também são objetos
- Cada instância de string tem seus próprios atributos e métodos
- O atributo mais importante da string é a sua coleção de caracteres
- Há uma grande variedade de métodos
- Teste o exemplo:

```
ss = "Hello, World"
print(ss.upper())
tt = ss.lower()
print(tt)
```

Métodos de Strings

- Outros métodos para você testar:

Método	Parâmetros	Descrição
upper	nenhum	Retorna um string todo em maiúsculas
lower	nenhum	Retorna um string todo em minúsculas
capitalize	nenhum	Retorna um string com o primeiro caractere em maiúscula, e o resto em minúsculas
strip	nenhum	Retorna um string removendo caracteres em branco do início e do fim
lstrip	nenhum	Retorna um string removendo caracteres em brando do início
rstrip	nenhum	Retorna um string removendo caracteres em brando do fim
count	item	Retorna o número de ocorrências de item
replace	old, new	Substitui todas as ocorrências do substring old por new
center	largura	Retorna um string centrado em um campo de tamanho largura
ljust	largura	Retorna um string justificado à esquerda em um campo de tamanho largura
rjust	largura	Retorna um string justificado à direita em um campo de tamanho largura
find	item	Retorna o índice mais à esquerda onde o substring item é encontrado
rfind	item	Retorna o índice mais à direita onde o substring item é encontrado
index	item	Como find, mas causa um erro em tempo de execução caso item não seja encontrado
rindex	item	Como rfind, mas causa um erro em tempo de execução caso item não seja encontrado

Métodos de Strings

- Teste o exemplo:

```
ss = "    Hello, World    "

els = ss.count("l")
print(els)

print("***"+ss.strip()+"***")
print("***"+ss.lstrip()+"***")
print("***"+ss.rstrip()+"***")

news = ss.replace("o", "***")
print(news)
```

Métodos de Strings

- Outro exemplo:

```
food = "banana bread"
print(food.capitalize())

print("*"+food.center(25)+"*")
print("*"+food.ljust(25)+"*")
print("*" +food.rjust(25)+"*")

print(food.find("e"))
print(food.find("na"))
print(food.find("b"))

print(food.rfind("e"))
print(food.rfind("na"))
print(food.rfind("b"))

print(food.index("e"))
```

Métodos de Strings

- Existem ainda muitos outros métodos úteis: isalnum(), isalpha(), isdigit(), islower(), isspace(), isupper(), lstrip(), replace(), strip(), entre outros Faça os testes!!!!

- Teste com os valores Araraquara e depois 1234:

```
teste_str=input("Digite um valor: ")
print(teste_str, " capitalize",teste_str.capitalize())
print(teste_str," center",teste_str.center(20))
print(teste_str," count",teste_str.count("a"))
print(teste_str," find",teste_str.find("ara"))
print(teste_str," isalnum",teste_str.isalnum())
print(teste_str," isalpha",teste_str.isalpha())
print(teste_str," isdigit",teste_str.isdigit())
print(teste_str," replace",teste_str.replace("a","o"))
```

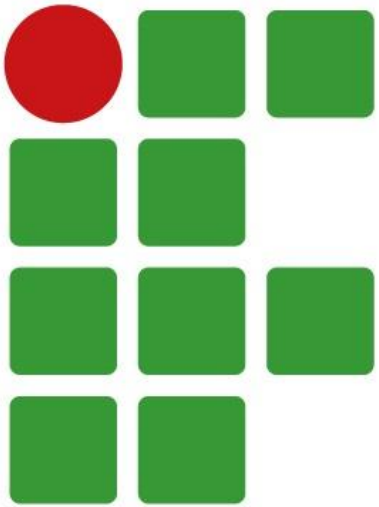
Exercícios

1. Escreva um programa que remove a primeira ocorrência de uma letra de uma string. A string e a letra devem ser fornecidas pelo usuário.
2. Escreva um programa que remove todas as ocorrências de uma letra de uma string. A string e a letra devem ser fornecidas pelo usuário.
3. Escreva um programa que verifique se duas strings fornecidas pelo usuário são iguais e mostre o total de caracteres de cada uma delas. Diferencie letras maiúsculas das minúsculas.
4. Escreva um programa que reconhece se uma string é um palíndromo. Exemplo: arara, ovo, reter.
5. Faça um programa que recebe uma frase e retorna o número de palavras que a frase contém.
6. Faça um programa que solicite o nome do usuário e imprima-o na vertical e em formato de escada. Ex.:

```
F
FU
FUL
FULA
FULAN
FULANO
```

Exercícios

7. Faça um programa que permita ao usuário digitar o seu nome e em seguida o mostre de trás para frente utilizando somente letras maiúsculas.
8. Dada uma string com uma frase informada pelo usuário (incluindo espaços em branco), conte a quantidade de espaços em branco e a quantidade de vezes que aparecem as vogais a, e, i, o, u.
9. Um anagrama é uma palavra que é feita a partir da transposição das letras de outra palavra ou frase. Por exemplo, “Iracema” é um anagrama para “America”. Escreva um programa que decida se uma string é um anagrama de outra string, ignorando os espaços em branco. O programa deve considerar maiúsculas e minúsculas como sendo caracteres iguais, ou seja, “a” = “A”.
10. Escreva um programa que solicite ao usuário a entrada de um número inteiro positivo ou negativo e mostre a quantidade de dígitos desse número.



INSTITUTO FEDERAL
São Paulo
Câmpus São Carlos

Representação de Caracteres

- Um conhecimento importante para trabalhar com strings é a conhecer a sua forma de representação (*encoding*)

- Uma grande diferença entre as versões 2.X e 3.X do Python é que, na versão 2.X as strings são codificados em ASCII e na versão 3.X em UTF-8

- No ASCII (*American Standard Code for Information Interchange*), cada caractere é representado usando apenas 7 bits, sendo que os caracteres de 0 a 31 são considerados de controle, e os de 32 a 127 correspondem aos caracteres comuns de um teclado de computador em inglês

Representação de Caracteres

- Teste o exemplo para ver os caracteres de 32 a 127. Observe que o número inteiro é convertido em caractere com chr():

```
print("Tabela ASCII de 32 a 127: ")
for i in range(32, 128, 3):
    print(f"ASCII[{i}] = {chr(i)}  |  ASCII[{i+1}] = {chr(i+1)}  |  ASCII[{i+2}] = {chr(i+2)}")
```

- A função chr converte um índice da tabela ASCII em seu caractere correspondente

Representação de Caracteres

- Observe que a ordem dos caracteres na tabela não é arbitrária: os caracteres de a a z (tanto minúsculos quanto maiúsculos) estão agrupados sequencialmente, respeitando a ordem alfabética

- Não há caracteres acentuados, pois estes não fazem parte da língua inglesa

- O Unicode é um conjunto de caracteres que inclui os símbolos de todas as línguas humanas, desde os milhares de caracteres de línguas asiáticas como o chinês, até os caracteres de línguas mortas como os hieróglifos usados pelos egípicios.

Representação de Caracteres

- Para representar os caracteres do Unicode, foram desenvolvidos diversas formas de codificação. Uma delas é a UTF-32 (U de *Universal Character Set* e o TF de *Transformation Format* - usando 32 bits)

- Essa forma utiliza sempre 32 bits (ou 4 bytes) para codificar um caractere (ou, mais exatamente, para codificar o índice do caractere na tabela Unicode)

- O UTF-32 não é muito utilizado hoje pois, apesar de ser simples de decodificar, seus arquivos tornam-se muito grandes

Representação de Caracteres

- O UTF-16 exige ao menos 2 bytes (16 bits) para codificar cada caractere, criando arquivos menores em geral. Esse padrão é utilizado na linguagem Java e em sistemas operacionais como o Microsoft Windows e o Mac OS X

- Já o padrão mais utilizado para codificar páginas na Internet atualmente é o UTF-8, que usa pelo menos 8 bits para representar cada caractere